Documento de Análisis y Diseño - Proyecto CinemaUQ

MARIANA VASQUEZ ZAPATA

1. Actores del Sistema

Para este proyecto, se identificaron dos tipos de usuarios principales que interactúan con la plataforma:

- **Cliente:** Es el usuario final que se registra en la aplicación. Su rol principal es consultar la cartelera, elegir qué película ver, seleccionar sus asientos y comprar entradas junto con productos de confitería. Además, administra su propia tarjeta virtual, acumula y redime puntos, y gestiona el historial o las cancelaciones de sus compras.
- **Administrador:** Es el usuario encargado de la gestión integral del cine. Tiene privilegios para crear y modificar películas, configurar las salas y funciones, gestionar el inventario de confitería y promociones, recargar las tarjetas de los clientes y generar los reportes operativos y visuales del negocio.

2. Requisitos Funcionales (RF)

Se han codificado los requisitos funcionales mínimos que la plataforma debe cumplir para operar correctamente:

- **RF-001:** Permitir el registro de usuarios y el inicio de sesión para diferenciar si quien ingresa es un Cliente o el Administrador.
- **RF-002:** Permitir al administrador registrar, editar y configurar las películas, las salas de cine y la distribución física de sus asientos.
- **RF-003:** Programar nuevas funciones vinculando una película específica con una sala, en una fecha y hora determinadas.
- **RF-004:** Permitir al cliente explorar la cartelera y filtrar las funciones disponibles por fecha, hora o sala.
- **RF-005:** Mostrar una interfaz gráfica con la disponibilidad en tiempo real de los asientos (disponibles, ocupados, seleccionados, accesibles) para evitar vender el mismo asiento dos veces.
- **RF-006:** Permitir al cliente agregar productos individuales y combos de confitería a su carrito de compras.
- **RF-007:** Procesar los pagos exclusivamente a través de la tarjeta virtual del cliente, validando primero que esté activa y que el saldo sea suficiente para cubrir el total.
- **RF-008:** Permitir al administrador realizar recargas de saldo en las tarjetas virtuales de los clientes, guardando el registro de la transacción.

- **RF-009:** Gestionar cancelaciones de compras y aplicar reembolsos automáticos a la tarjeta según las políticas de tiempo (devolución del 100% si faltan más de 24h, 80% si faltan entre 2h y 24h, y sin reembolso si faltan menos de 2 horas).
- **RF-010:** Calcular y asignar "Puntos CinemaUQ" por cada compra, controlando que tengan exactamente un año de vigencia y que, al usarse, se descuenten primero los más antiguos.
- **RF-011:** Permitir que el cliente use sus puntos acumulados para pagar o canjear entradas y productos de confitería.
- **RF-012:** Generar reportes para el administrador, tanto exportables (PDF o TXT) como gráficos (diagramas de barras, pastel y líneas usando JavaFX) para analizar ventas, ocupación y recargas.

3. Reglas de Negocio Clave

Para garantizar que la lógica de la aplicación sea consistente, se establecieron las siguientes reglas:

- **RN-001 (Control de saldo):** No se puede concretar ninguna transacción si el costo total (entradas + comida + tarifa de servicio) supera el saldo de la tarjeta virtual, o si esta se encuentra inactiva.
- **RN-002 (Bloqueo de asientos):** En el instante en que se confirma y paga una compra, los asientos reservados pasan a estado Ocupado y desaparecen de la disponibilidad para el resto de los usuarios en esa función.
- **RN-003 (Política de reembolsos):** El sistema bloquea cualquier intento de cancelación del cliente si faltan menos de 2 horas para la función. Si la cancelación procede (100% o 80%), el dinero solo se devuelve a la tarjeta virtual de origen. Adicionalmente, si la función es cancelada directamente por el cine (administrador), se devolverá automáticamente el 100% del valor pagado a todos los clientes. Ninguna compra puede ser reembolsada dos veces.
- **RN-004 (Dinámica de acumulación de puntos):** Por cada 10.000 COP gastados en el sistema, el cliente recibe 10 puntos. Estos vencen a los 365 días exactos desde su fecha de obtención. Para redimirlos, el sistema ejecuta un comportamiento FIFO (First In, First Out), gastando primero los puntos próximos a vencer.
- **RN-005 (Equivalencia y Redención de puntos):** El sistema permite el pago mixto o total con puntos bajo la siguiente tabla de equivalencias:

Beneficio / Producto	**Costo en Puntos**
Entrada General (1 Asiento)	500 Puntos
Producto Individual (Snack/Bebida)	200 Puntos
Combo de Confitería	600 Puntos

- **RN-006 (Control de salas):** El sistema impide que se programen dos funciones en la misma sala si sus horarios se superponen.

4. Casos de Uso Principales

1. **Comprar Entradas y Confitería (Cliente):** El usuario navega la cartelera, elige una función, selecciona sus asientos de forma gráfica, añade un combo de palomitas y finaliza el flujo pagando con el saldo de su tarjeta virtual.
2. **Recargar Tarjeta Virtual (Administrador):** Un cliente se da cuenta de que no tiene saldo y solicita una recarga. El administrador busca la tarjeta en el sistema, ingresa el monto y la plataforma actualiza el saldo registrando el movimiento.
3. **Cancelar Compra (Cliente/Administrador):** Un usuario decide que no puede asistir. El sistema revisa la hora actual contra la hora de la función, calcula si le corresponde el 100% o el 80% de devolución, y retorna ese dinero a su tarjeta.
4. **Redimir Puntos (Cliente):** Al momento de pagar, el cliente decide usar sus puntos acumulados. El sistema valida su saldo de puntos vigentes, descuenta los necesarios según la tabla de equivalencias y elimina/reduce los lotes de puntos más antiguos.

5. Modelo del Dominio (Entidades y Relaciones)

Tomando como base el diagrama UML diseñado para la implementación, las clases principales y cómo se relacionan quedaron definidas de la siguiente manera:

- Se cuenta con una clase abstracta `Usuario` de la cual heredan `Cliente` y `Administrador`.
- Cada `Cliente` tiene una relación de 1 a 1 con su `TarjetaVirtual`, y esta a su vez contiene una lista de `MovimientoTarjeta` (1 a muchos).
- El cine se organiza así: una `Pelicula` tiene muchas `Funcion`es (1 a muchos), y una `Sala` también tiene muchas `Funcion`es asignadas (1 a muchos).
- A nivel físico, una `Sala` está compuesta por muchos `Asiento`s (1 a muchos).
- Cuando un cliente compra, se genera una `Compra`. Esta clase central agrupa al `Cliente`, la `Funcion`, múltiples `Boleta`s (que reservan los asientos) y los `DetalleConfiteria`.
- Para los productos, se utiliza una clase abstracta `ProductoConfiteria` que se divide en `ProductoIndividual` y `Combo`.
- Finalmente, para la fidelización, el cliente tiene un registro de `PuntosLealtad` compuesto por múltiples `LotePuntos` para poder controlar las fechas de vencimiento de cada ganancia.

6. Justificación de Principios SOLID

El código ha sido estructurado para cumplir estrictamente con los 5 principios de diseño:

- **Single Responsibility Principle (SRP):** Aplicado en la clase `CompraBuilder`. Su única responsabilidad es ensamblar y validar un objeto de compra paso a paso, dejando que la clase `Compra` solo se encargue de almacenar los datos y la clase `ProcesarCompraUseCase` se encargue de la lógica financiera.
- **Open/Closed Principle (OCP):** Aplicado en el sistema de reportes (`IExportadorReporte`). El sistema está abierto a la extensión (puedo crear un `ExportadorExcel` nuevo implementando la interfaz) pero cerrado a la modificación (no tengo que tocar el código de las vistas ni de los controladores para agregar un nuevo formato).
- **Liskov Substitution Principle (LSP):** Evidenciado en la jerarquía de confitería. Las clases `Combo` y `ProductoIndividual` pueden reemplazar a su clase base `ProductoConfiteria` en el cálculo del total de la compra sin romper el comportamiento del sistema ni lanzar excepciones inesperadas.
- **Interface Segregation Principle (ISP):** Implementado al separar las interfaces del negocio. El `IAutenticacionUseCase` es distinto del `IGestionPuntosUseCase` y de `IPagoTarjetaUseCase`. Así, el controlador de Login solo depende e inyecta la interfaz de autenticación, sin verse obligado a depender de métodos de recarga de tarjetas que no utiliza.
- **Dependency Inversion Principle (DIP):** Aplicado en la clase `ProcesarCompraUseCase`. Esta clase no instancia directamente (con `new`) la lógica de tarjetas ni de puntos, sino que recibe las abstracciones `IPagoTarjetaUseCase` e `IGestionPuntosUseCase` a través de su constructor, invirtiendo la dependencia hacia las interfaces.

7. Patrones de Diseño

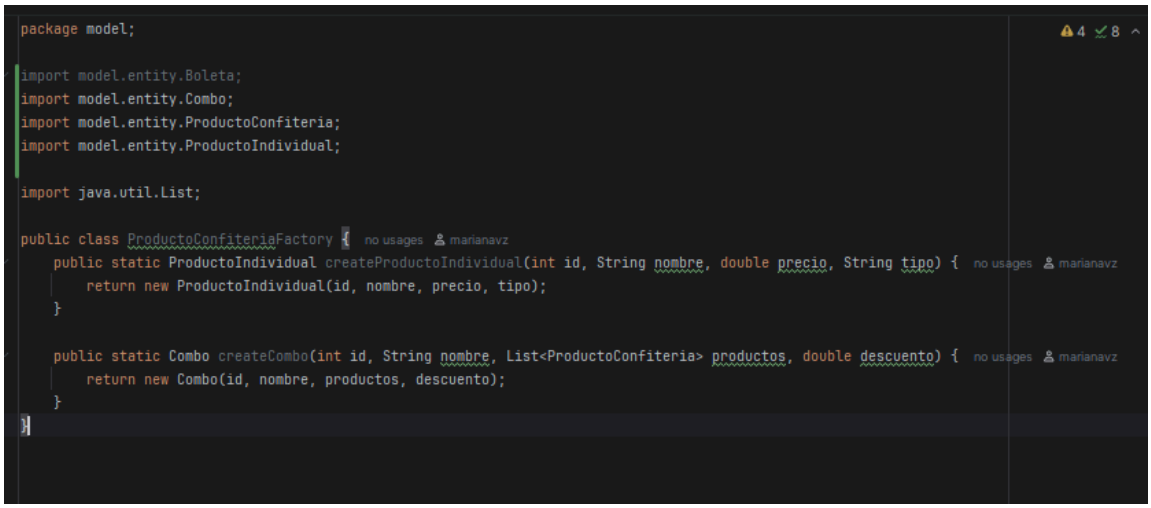
A continuación, se documentan los patrones implementados para resolver problemas específicos del proyecto, categorizados según su tipo.

Patrones Creacionales (3)

1. Factory Method (Para la confitería)

- **Problema identificado:** Crear productos es complejo porque un "Snack" individual no se inicializa igual que un "Combo" (que incluye varios productos y aplica un porcentaje de descuento). Llenar el código de if/else o exponer la lógica de instanciación en la vista genera un alto acoplamiento.
- **Patrón utilizado:** Factory Method.
- **Justificación:** Centraliza la creación de los productos. Si mañana el cine inventa un nuevo tipo de producto, solo se agrega un método a la fábrica sin romper la lógica del carrito de compras en la interfaz gráfica.
- **Clases involucradas:** `ProductoConfiteriaFactory`, `ProductoIndividual`, `Combo`, `ProductoConfiteria`.

- **Evidencia de implementación:**



```java
package model;

import model.entity.Boleta;
import model.entity.Combo;
import model.entity.ProductoConfiteria;
import model.entity.ProductoIndividual;

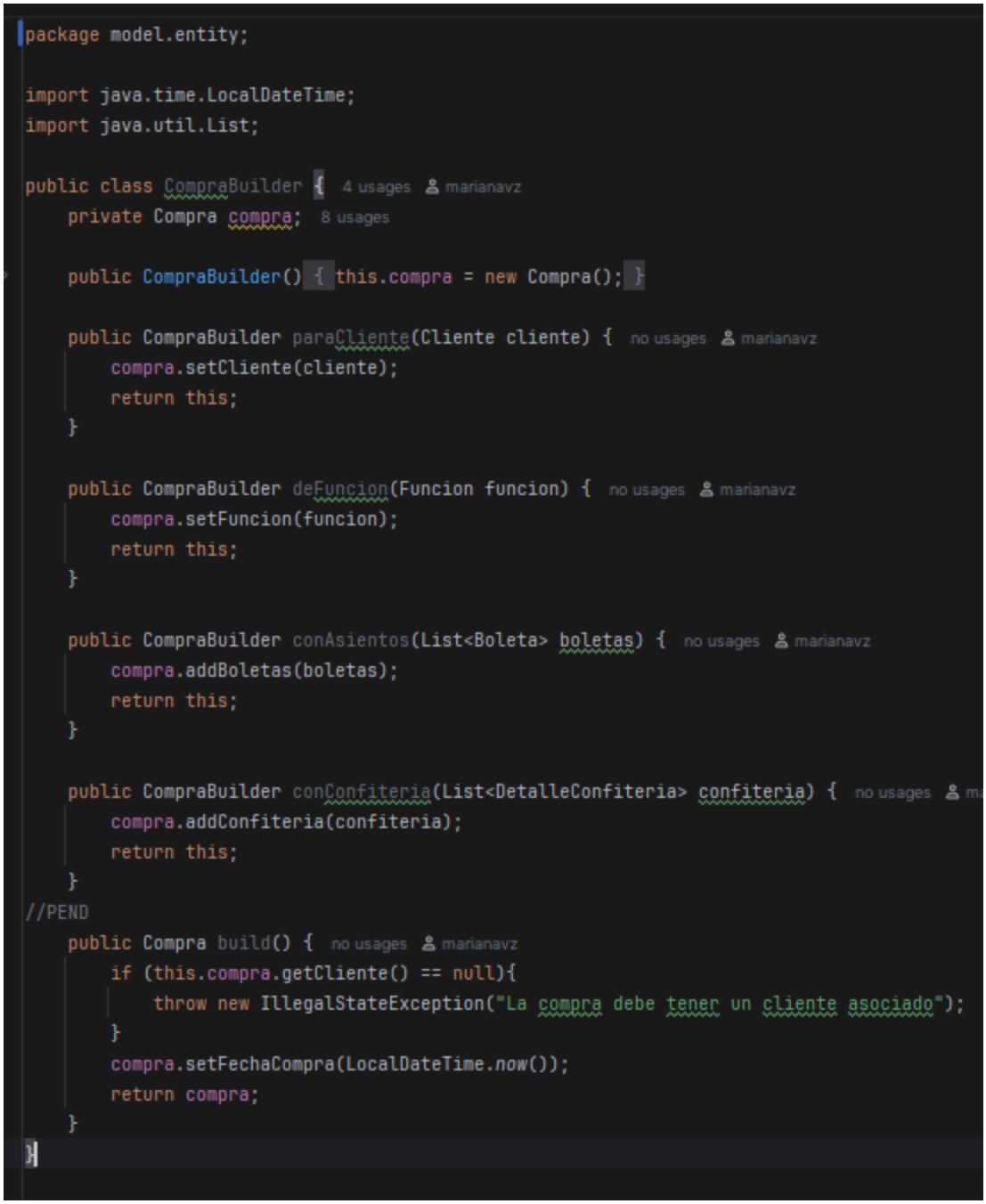
import java.util.List;

public class ProductoConfiteriaFactory {
    public static ProductoIndividual createProductoIndividual(int id, String nombre, double precio, String tipo) {
        return new ProductoIndividual(id, nombre, precio, tipo);
    }

    public static Combo createCombo(int id, String nombre, List<ProductoConfiteria> productos, double descuento) {
        return new Combo(id, nombre, productos, descuento);
    }
}
```

2. Builder (Para armar las compras)

- **Problema identificado:** Construir un objeto `Compra` requería pasar demasiados parámetros obligatorios y opcionales al constructor (cliente, función, boletas, confitería, fecha), creando un "telescoping constructor" propenso a errores de posición de variables.
- **Patrón utilizado:** Builder.
- **Justificación:** Permite armar la compra de forma fluida y paso a paso. El método final `build()` asegura que el objeto nazca en un estado válido (por ejemplo, validando que la compra tenga un cliente asociado antes de existir).
- **Clases involucradas:** `CompraBuilder`, `Compra`.
- **Evidencia de implementación:**

- 

```java
package model.entity;

import java.time.LocalDateTime;
import java.util.List;

public class CompraBuilder {  4 usages  marianavz
    private Compra compra;  8 usages

    public CompraBuilder() { this.compra = new Compra(); }

    public CompraBuilder paraCliente(Cliente cliente) {  no usages  marianavz
        compra.setCliente(cliente);
        return this;
    }

    public CompraBuilder deFuncion(Funcion funcion) {  no usages  marianavz
        compra.setFuncion(funcion);
        return this;
    }

    public CompraBuilder conAsientos(List<Boleta> boletas) {  no usages  marianavz
        compra.addBoletas(boletas);
        return this;
    }

    public CompraBuilder conConfiteria(List<DetalleConfiteria> confiteria) {  no usages  m
        compra.addConfiteria(confiteria);
        return this;
    }
//PEND
    public Compra build() {  no usages  marianavz
        if (this.compra.getCliente() == null){
            throw new IllegalStateException("La compra debe tener un cliente asociado");
        }
        compra.setFechaCompra(LocalDateTime.now());
        return compra;
    }
}
```

3. Singleton (Para la sesión del sistema)

- **Problema identificado:** Se necesitaba una forma de saber globalmente quién es el usuario autenticado en ese momento, sin tener que pasar el objeto "Usuario" como parámetro a través de los constructores de cada ventana de JavaFX.
- **Patrón utilizado:** Singleton.
- **Justificación:** Garantiza una única instancia compartida del contexto de la aplicación. Cualquier controlador puede consultar

`CinemaUQContext.getInstance().getCurrentUser()` para saber si debe mostrar las vistas de Administrador o Cliente.

- **Clases involucradas:** `CinemaUQContext`.
- **Evidencia de implementación:**
-

```java
package model;

import model.entity.Usuario;

public class CinemaUQContext {  7 usages  marianavz
    private static CinemaUQContext instance;  3 usages
    private Usuario currentUser;  2 usages
//singleton
    private CinemaUQContext() {  1 usage  marianavz
        // Constructor privado para evitar instanciación externa
    }

    public static CinemaUQContext getInstance() {  2 usages  marianavz
        if (instance == null) {
            instance = new CinemaUQContext();
        }
        return instance;
    }

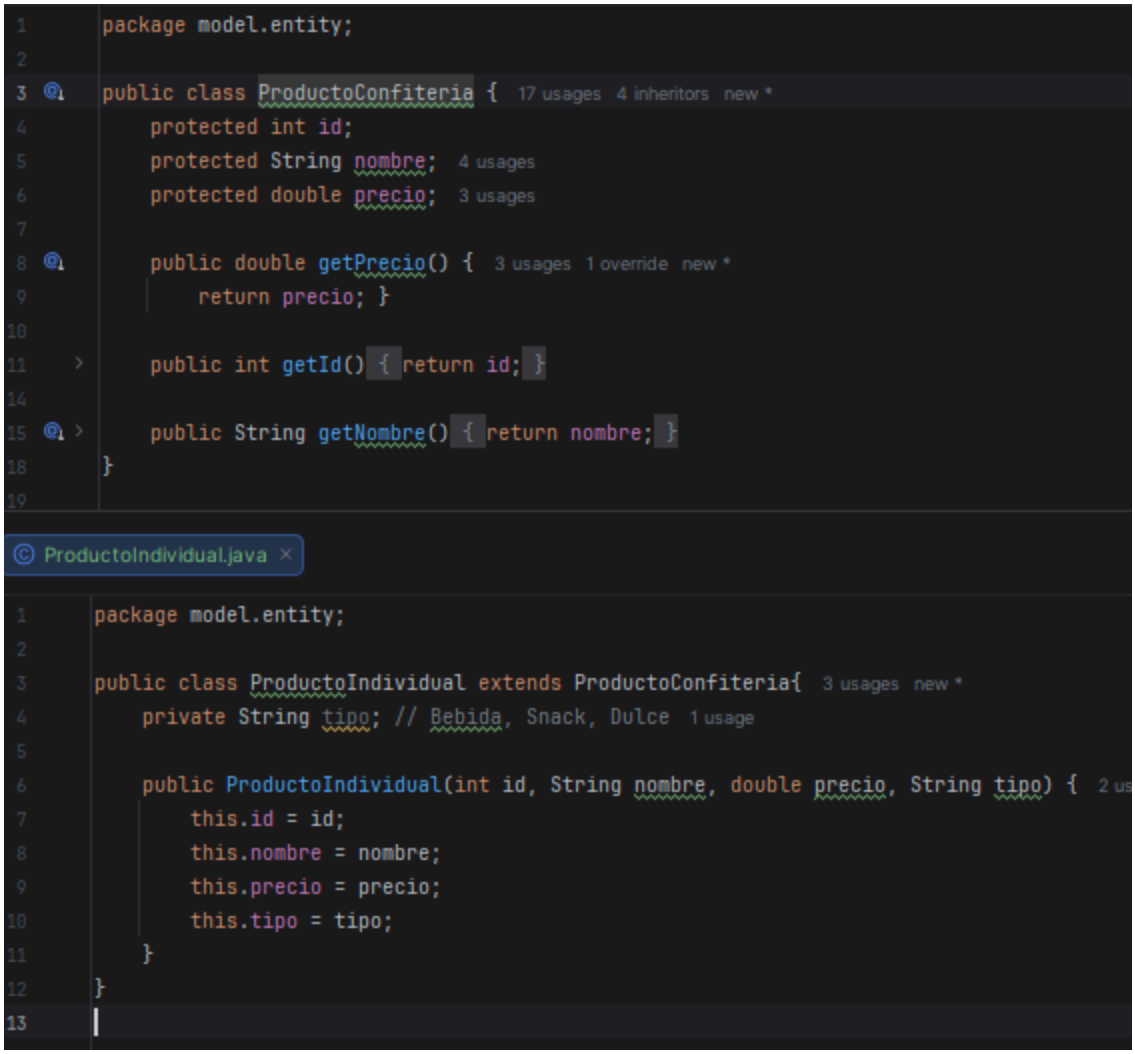
    public Usuario getCurrentUser() { return currentUser; }

    public void setCurrentUser(Usuario usuario) { this.currentUser = usuario; }
}
```

Patrones Estructurales (4)

1. Composite (Estructura de Confitería)

- **Problema identificado:** El carrito de compras necesitaba calcular el total de los productos de confitería, pero un `Combo` está compuesto a su vez por múltiples Productos Individuales, lo que obligaba a hacer validaciones de tipo (instanceof) para sumar los precios.
- **Patrón utilizado:** Composite.
- **Justificación:** Permite estructurar objetos en forma de árbol. Al hacer que `Combo` herede de `ProductoConfiteria` y a la vez contenga una lista de ellos, el sistema trata productos simples y compuestos de forma idéntica al llamar a `getPrecio()`.
- **Clases involucradas:** `ProductoConfiteria` (Componente), `ProductoIndividual` (Hoja), `Combo` (Compuesto).
- **Evidencia de implementación:**

- 

2. Decorator (Adiciones dinámicas)

- **Problema identificado:** Los clientes desean agregar características adicionales a sus productos (como "Extra Queso" en los nachos o "Agrandar" una bebida) modificando su precio y descripción, pero crear subclases para cada variación (`NachosConQueso`, `NachosSinQueso`) genera una explosión de clases.
- **Patrón utilizado:** Decorator.
- **Justificación:** Permite añadir responsabilidades a objetos individuales de forma dinámica en tiempo de ejecución. El decorador envuelve el producto base sumando el costo del extra sin modificar la clase original.
- **Clases involucradas:** `ProductoConfiteriaDecorator`, `AdicionQuesoDecorator`.
- **Evidencia de implementación:**

```java
package model.decorator;

import model.entity.ProductoConfiteria;

public abstract class ProductoConfiteriaDecorator extends ProductoConfiteria {  1 usage  1 inhe
    protected ProductoConfiteria productoBase;  3 usages

    public ProductoConfiteriaDecorator(ProductoConfiteria productoBase) {  1 usage  new *
        this.productoBase = productoBase;
        this.id = productoBase.getId();
        // Hereda el nombre base inicialmente
        this.nombre = productoBase.getNombre();
    }
}
```

AdicionQuesoDecorator.java

```java
package model.decorator;

import model.entity.ProductoConfiteria;

public class AdicionQuesoDecorator extends ProductoConfiteriaDecorator {  no usages  new *

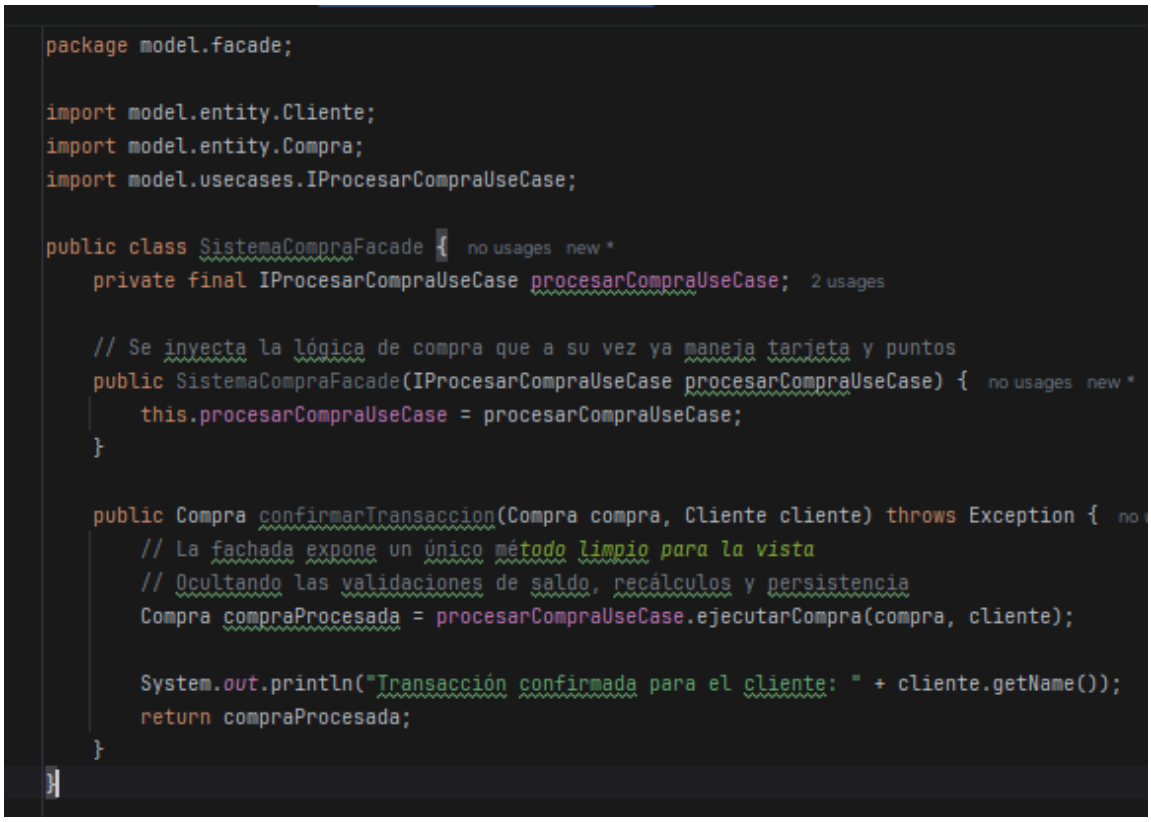
    public AdicionQuesoDecorator(ProductoConfiteria productoBase) { super(productoBase); }

    @Override  3 usages  new *
    public double getPrecio() { return productoBase.getPrecio() + 2500.0; }

    @Override  2 usages  new *
    public String getNombre() { return productoBase.getNombre() + " con Extra Queso"; }
}
```

3. Facade (Fachada de Compras)

- **Problema identificado:** Los controladores de la vista (JavaFX) tenían que interactuar con la validación de saldos, debitar dinero, calcular totales y asignar puntos, generando un alto acoplamiento entre la interfaz y la lógica compleja de negocio.
- **Patrón utilizado:** Facade.
- **Justificación:** Proporciona una interfaz simplificada de alto nivel. La vista solo llama a `confirmarTransaccion()` en la fachada, y esta se encarga de orquestar por debajo todos los casos de uso necesarios (pagos y puntos).
- **Clases involucradas:** `SistemaCompraFacade`, `IProcesarCompraUseCase`.
- **Evidencia de implementación:**

- 

```java
package model.facade;

import model.entity.Cliente;
import model.entity.Compra;
import model.usecases.IProcesarCompraUseCase;

public class SistemaCompraFacade {  no usages  new *
    private final IProcesarCompraUseCase procesarCompraUseCase;  2 usages

    // Se inyecta la lógica de compra que a su vez ya maneja tarjeta y puntos
    public SistemaCompraFacade(IProcesarCompraUseCase procesarCompraUseCase) {  no usages  new *
        this.procesarCompraUseCase = procesarCompraUseCase;
    }

    public Compra confirmarTransaccion(Compra compra, Cliente cliente) throws Exception {  no
        // La fachada expone un único método limpio para la vista
        // Ocultando las validaciones de saldo, recálculos y persistencia
        Compra compraProcesada = procesarCompraUseCase.ejecutarCompra(compra, cliente);

        System.out.println("Transacción confirmada para el cliente: " + cliente.getName());
        return compraProcesada;
    }
}
```

4. Proxy (Seguridad de Tarjetas)

- **Problema identificado:** El método para recargar las tarjetas virtuales residía en el mismo servicio que debita el saldo. Se corría el riesgo de que cualquier parte del código pudiera invocar una recarga sin validar los permisos.
- **Patrón utilizado:** Protection Proxy.
- **Justificación:** El Proxy envuelve el servicio real. intercepta la petición de recarga, verifica en el `CinemaUQContext` si el usuario actual es de tipo Administrador, y solo si es así, permite que la transacción se ejecute en el objeto real.
- **Clases involucradas:** `PagoTarjetaUseCaseProxy`, `PagoTarjetaUseCase`, `IPagoTarjetaUseCase`.
- **Evidencia de implementación:**

```java
package model.usecases;

import model.entity.Administrador;
import model.CinemaUQContext;
import model.entity.Cliente;

public class PagoTarjetaUseCaseProxy implements IPagoTarjetaUseCase {  no usages  new *

    private final PagoTarjetaUseCase servicioReal;  4 usages

    public PagoTarjetaUseCaseProxy(PagoTarjetaUseCase servicioReal) { this.servicioReal = servicioReal; }

    @Override  2 usages  new *
    public boolean verificarSaldoSuficiente(Cliente cliente, double montoTotal) {
        // Método de acceso público, el proxy simplemente delega la ejecución
        return servicioReal.verificarSaldoSuficiente(cliente, montoTotal);
    }

    @Override  2 usages  new *
    public void debitarSaldo(Cliente cliente, double montoTotal) throws Exception {
        // Método de acceso público para procesar compras
        servicioReal.debitarSaldo(cliente, montoTotal);
    }

    @Override  1 usage  new *
    public void recargarTarjeta(Cliente cliente, double monto) throws Exception {
        // Lógica del Proxy: Control de acceso estricto
        var usuarioActual = CinemaUQContext.getInstance().getCurrentUser();

        if (usuarioActual instanceof Administrador) {
            servicioReal.recargarTarjeta(cliente, monto);
            System.out.println("Recarga exitosa aprobada por el administrador.");
        } else {
            throw new SecurityException("Acceso denegado: Solo un Administrador autenticado puede realizar recargas de saldo.");
        }
    }
}
```

-
-